PROJECT REPORT

LIBRARY MANAGEMENT SYSTEM (LMS)

A Relational Database Management System (RDBMS) Implementation with Stored Procedures & Triggers

1. Problem Statement

Managing a library in an academic institution using manual ledgers or decentralized flat files creates severe operational bottlenecks. Without a centralized relational database, administrators face data redundancy, inconsistent inventory tracking, and inaccurate calculation of late return penalties.

Key operational challenges in legacy manual circulation systems include:

• Inventory Discrepancies: When multiple students borrow or return books simultaneously, manual record-keeping fails to update stock availability reliably, leading to out-of-stock errors and missing copies.

• Manual Fine Computation: Late fee penalties calculated by staff are prone to human error and inconsistency, causing disputes with borrowers regarding return due dates.

• Lack of Referential Integrity: Without primary and foreign key constraints, deleting an author, category, or member can orphan historical borrowing records and destroy institutional audit trails.

• Complex Reporting: Answering basic administrative questions—such as identifying unreturned books, auditing fine collections, or viewing books by category—requires tedious cross-referencing across separate physical ledgers.

Solution Objective: This project develops a relational Library Management System (LMS) database using MySQL. By structuring data into normalized tables (Category, Author, Book, Member, Loan, Fine) and implementing automated Stored Procedures (AddMember, BorrowBook) and Database Triggers (FineTrigger), the system ensures real-time inventory synchronization, ACID compliance, and automated overdue penalty calculations at ₹10 per day.

2. Technology Stack

The project is architected using a modern relational database ecosystem designed for high data integrity, modular procedure execution, and standardized SQL querying:

Layer / Component	Technology & Tool Selected
Database Engine	MySQL 8.0 Server / RDBMS
Storage Engine	InnoDB (Supports Referential Integrity, Foreign Keys, Row-Level Locking)
Query Language	Structured Query Language (SQL DDL, DML, DQL)
Database Programming	MySQL Stored Procedures (CALL), Event-Driven Database Triggers (AFTER UPDATE)
Client & Command Interface	MySQL Command Line Client / Terminal Monitor
Operating System Environment	Cross-Platform (Windows / Linux / macOS compatible)

3. Source Code

This section presents the exact SQL source code implemented for creating the schema tables, seeding initial inventory and member records, and automating workflows via Stored Procedures and Triggers.

3.1 Data Definition Language (DDL) Script

```
-- Create Database and Switch Context
CREATE DATABASE LibraryManagementSystem;
USE LibraryManagementSystem;

-- 1. Category Table
CREATE TABLE Category(
    CategoryID INT PRIMARY KEY AUTO_INCREMENT,
    CategoryName VARCHAR(50)
);

-- 2. Author Table
CREATE TABLE Author(
    AuthorID INT PRIMARY KEY AUTO_INCREMENT,
    AuthorName VARCHAR(100)
);

-- 3. Book Table
CREATE TABLE Book(
    BookID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(100),
    AuthorID INT,
    CategoryID INT,
```

```
    Quantity INT,
    FOREIGN KEY(AuthorID) REFERENCES Author(AuthorID),
    FOREIGN KEY(CategoryID) REFERENCES Category(CategoryID)
);

-- 4. Member Table
CREATE TABLE Member(
    MemberID INT PRIMARY KEY AUTO_INCREMENT,
    MemberName VARCHAR(100),
    Phone VARCHAR(15),
    Email VARCHAR(100)
);

-- 5. Loan Table
CREATE TABLE Loan(
    LoanID INT PRIMARY KEY AUTO_INCREMENT,
    BookID INT,
    MemberID INT,
    IssueDate DATE,
    DueDate DATE,
    ReturnDate DATE,
    FOREIGN KEY(BookID) REFERENCES Book(BookID),
    FOREIGN KEY(MemberID) REFERENCES Member(MemberID)
);

-- 6. Fine Table
CREATE TABLE Fine(
    FineID INT PRIMARY KEY AUTO_INCREMENT,
    LoanID INT,
    FineAmount DECIMAL(10,2),
    FOREIGN KEY(LoanID) REFERENCES Loan(LoanID)
);
```

3.2 Data Manipulation Language (DML) Seed Script

```
-- Insert Categories
INSERT INTO Category(CategoryName)
VALUES
('Programming'),
('Database'),
('Science'),
('Mathematics');

-- Insert Authors
INSERT INTO Author(AuthorName)
VALUES
('James Gosling'),
```

```
('Dennis Ritchie'),
('C J Date'),
('Bjarne Stroustrup');

-- Insert Books
INSERT INTO Book(Title,AuthorID,CategoryID,Quantity)
VALUES
('Java Programming',1,1,5),
('C Programming',2,1,4),
('Database System',3,2,6),
('C++ Programming',4,1,3);

-- Insert Members
INSERT INTO Member(MemberName,Phone,Email)
VALUES
('Shailesh','9876543210','meshailesh003@gmail.com'),
('Sandeep','9876543211','sandeep123@gmail.com'),
('Sanjiv','9876543212','sanjiv01@gmail.com');

-- Insert Initial Loan Records
INSERT INTO Loan(BookID,MemberID,IssueDate,DueDate,ReturnDate)
VALUES
(1,1,'2026-07-01','2026-07-10','2026-07-09'),
(2,2,'2026-07-03','2026-07-12','2026-07-15'),
(3,3,'2026-07-05','2026-07-14',NULL);

-- Stock Sync for Loaned Books
UPDATE Book SET Quantity=Quantity-1 WHERE BookID=1;
UPDATE Loan SET ReturnDate=CURDATE() WHERE LoanID=3;
UPDATE Book SET Quantity=Quantity+1 WHERE BookID=3;
```

3.3 Stored Procedures & Triggers Script

```
-- Stored Procedure: AddMember (Automated Registration)
DELIMITER //
CREATE PROCEDURE AddMember(
    IN mname VARCHAR(100),
    IN phone VARCHAR(15),
    IN email VARCHAR(100)
)
BEGIN
    INSERT INTO Member(MemberName,Phone,Email)
    VALUES(mname,phone,email);
END //
DELIMITER ;

-- Stored Procedure: BorrowBook (Automated Checkout & Stock Decrement)
```

```
DELIMITER //
CREATE PROCEDURE BorrowBook(
    IN bID INT,
    IN mID INT,
    IN issue DATE,
    IN due DATE
)
BEGIN
    INSERT INTO Loan(BookID,MemberID,IssueDate,DueDate)
    VALUES(bID,mID,issue,due);

    UPDATE Book SET Quantity=Quantity-1 WHERE BookID=bID;
END //
DELIMITER ;

-- Database Trigger: FineTrigger (Automated Late Fee Calculation at Rs. 10/day)
DELIMITER //
CREATE TRIGGER FineTrigger
AFTER UPDATE ON Loan
FOR EACH ROW
BEGIN
    IF NEW.ReturnDate IS NOT NULL AND NEW.ReturnDate>NEW.DueDate THEN
        INSERT INTO Fine(LoanID,FineAmount)
        VALUES(
            NEW.LoanID,
            DATEDIFF(NEW.ReturnDate, NEW.DueDate)*10
        );
    END IF;
END //
DELIMITER ;
```

4. System Output & Demonstration

The following subsections display the exact terminal logs and query results captured during live MySQL session testing, confirming database operations, procedure executions, trigger fine generations, and relational reporting.

4.1 Stored Procedure Execution Logs (AddMember & BorrowBook)

The logs below demonstrate calling the AddMember procedure to register 'Amit' and executing BorrowBook for member 1 borrowing 'C Programming' (BookID 2):

```
mysql> CALL AddMember(
    -> 'Amit',
    -> '9876543222',
    -> 'amit@gmail.com'
```

```
    -> );
Query OK, 1 row affected (0.03 sec)

mysql> CALL BorrowBook(
    -> 2,
    -> 1,
    -> '2026-07-20',
    -> '2026-07-30'
    -> );
Query OK, 1 row affected (0.03 sec)
```

4.2 Trigger Verification Log (Automated Fine Generation)

When LoanID 2 is updated with a ReturnDate of '2026-07-18' (6 days past the DueDate of '2026-07-12'), the FineTrigger fires automatically, calculating a fine of Rs. 60.00 (6 days * Rs. 10):

```
mysql> UPDATE Loan
    -> SET ReturnDate='2026-07-18'
    -> WHERE LoanID=2;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

4.3 Comprehensive Circulation Audit (Loan Status Query)

Executing a multi-table JOIN query displays the complete borrowing history, linking Loan records with Book titles and Member names:

```
mysql> SELECT
    -> LoanID,
    -> Title,
    -> MemberName,
    -> IssueDate,
    -> DueDate,
    -> ReturnDate
    -> FROM Loan
    -> JOIN Book
    -> ON Loan.BookID=Book.BookID
    -> JOIN Member
    -> ON Loan.MemberID=Member.MemberID;
+--------+------------------+------------+------------+------------+------------+
| LoanID | Title            | MemberName | IssueDate  | DueDate    | ReturnDate |
+--------+------------------+------------+------------+------------+------------+
|      1 | Java Programming | Shailesh   | 2026-07-01 | 2026-07-10 | 2026-07-09 |
|      2 | C Programming    | Sandeep    | 2026-07-03 | 2026-07-12 | 2026-07-18 |
```

```
|      3 | Database System  | Sanjiv     | 2026-07-05 | 2026-07-14 | 2026-07-24 |
|      4 | C Programming    | Shailesh   | 2026-07-20 | 2026-07-30 | NULL       |
+--------+------------------+------------+------------+------------+------------+
4 rows in set (0.02 sec)
```

4.4 Book Inventory & Stock Availability Verification

Querying the Book table verifies that available quantities have correctly updated following circulation procedures:

```
mysql> SELECT * FROM Book;
+--------+------------------+----------+------------+----------+
| BookID | Title            | AuthorID | CategoryID | Quantity |
+--------+------------------+----------+------------+----------+
|      1 | Java Programming |        1 |          1 |        4 |
|      2 | C Programming    |        2 |          1 |        3 |
|      3 | Database System  |        3 |          2 |        7 |
|      4 | C++ Programming  |        4 |          1 |        3 |
+--------+------------------+----------+------------+----------+
4 rows in set (0.00 sec)

mysql> SELECT Title,Quantity
    -> FROM Book;
+------------------+----------+
| Title            | Quantity |
+------------------+----------+
| Java Programming |        4 |
| C Programming    |        3 |
| Database System  |        7 |
| C++ Programming  |        3 |
+------------------+----------+
4 rows in set (0.00 sec)
```

4.5 Registered Member Directory Output

Querying the Member table confirms the enrollment of all initial students plus the newly added member 'Amit':

```
mysql> SELECT * FROM Member;
+----------+------------+------------+------------------------+
| MemberID | MemberName | Phone      | Email                  |
+----------+------------+------------+------------------------+
|        1 | Shailesh   | 9876543210 | meshailesh003@gmail.com |
|        2 | Sandeep    | 9876543211 | sandeep123@gmail.com    |
|        3 | Sanjiv     | 9876543212 | sanjiv01@gmail.com      |
```

```
|        4 | Amit       | 9876543222 | amit@gmail.com          |
+----------+------------+------------+-------------------------+
4 rows in set (0.00 sec)
```

4.6 Relational Mapping Outputs (Books by Author & Category)

The output below demonstrates relational JOIN operations mapping book inventory to respective Authors and academic Categories:

```
mysql> SELECT
    -> Title,
    -> AuthorName
    -> FROM Book
    -> JOIN Author
    -> ON Book.AuthorID=Author.AuthorID;
+------------------+-------------------+
| Title            | AuthorName        |
+------------------+-------------------+
| Java Programming | James Gosling     |
| C Programming    | Dennis Ritchie    |
| Database System  | C J Date          |
| C++ Programming  | Bjarne Stroustrup |
+------------------+-------------------+
4 rows in set (0.00 sec)

mysql> SELECT
    -> Title,
    -> CategoryName
    -> FROM Book
    -> JOIN Category
    -> ON Book.CategoryID=Category.CategoryID;
+------------------+--------------+
| Title            | CategoryName |
+------------------+--------------+
| Java Programming | Programming  |
| C Programming    | Programming  |
| Database System  | Database     |
| C++ Programming  | Programming  |
+------------------+--------------+
4 rows in set (0.00 sec)
```

4.7 Overdue Penalty & Total Fine Revenue Audit

Querying the Fine table verifies the exact penalty recorded by FineTrigger for LoanID 2, along with an aggregate query calculating total revenue:

```
mysql> SELECT
    -> LoanID,
    -> FineAmount
    -> FROM Fine;
+--------+------------+
| LoanID | FineAmount |
+--------+------------+
|      2 |      60.00 |
+--------+------------+
1 row in set (0.00 sec)

mysql> SELECT SUM(FineAmount)
    -> AS TotalFine
    -> FROM Fine;
+-----------+
| TotalFine |
+-----------+
|     60.00 |
+-----------+
1 row in set (0.02 sec)
```

4.8 Active Unreturned Books Query (Pending Loans)

The final query filters circulation records where ReturnDate is NULL, instantly identifying books currently checked out and unreturned:

```
mysql> SELECT
    -> Title,
    -> MemberName
    -> FROM Loan
    -> JOIN Book
    -> ON Loan.BookID=Book.BookID
    -> JOIN Member
    -> ON Loan.MemberID=Member.MemberID
    -> WHERE ReturnDate IS NULL;
+---------------+------------+
| Title         | MemberName |
+---------------+------------+
| C Programming | Shailesh   |
+---------------+------------+
1 row in set (0.00 sec)
```